

Proyecto: Gestor de Cursos con API, Favoritos y Seguridad SPA

1. Objetivo de la actividad

Desarrollar una aplicación web SPA en React que permita:

- Consumir datos desde una API externa.
- Mostrar información en componentes reutilizables.
- Buscar cursos por texto.
- Marcar cursos como favoritos.
- Guardar favoritos en `localStorage`.
- Utilizar `async/await`.
- Manejar datos en formato JSON.
- Aplicar buenas prácticas de seguridad frontend.
- Usar GitHub Copilot de forma responsable.
- Analizar el código con SonarQube.

2. Requisitos previos

Debes tener instalado:

- Node.js.
- npm.
- Visual Studio Code.
- Git.
- Navegador web.
- Extensión GitHub Copilot, opcional.
- Docker Desktop, recomendado para SonarQube local.

Verifica Node y npm:

```
node -v
npm -v
```

3. Crear proyecto React con Vite

Ejecuta:

```
npm create vite@latest gestor-cursos-react
```

Selecciona:

Framework: React



Variant: JavaScript

Luego entra al proyecto:

```
cd gestor-cursos-react
```

Instala dependencias:

```
npm install
```

Ejecuta el proyecto:

```
npm run dev
```

Normalmente se abrirá en:

```
http://localhost:5173
```

4. Instalar dependencias adicionales

Usaremos `axios` para consumir la API.

```
npm install axios
```

También instalaremos ESLint si deseas revisar calidad básica:

```
npm install -D eslint
```

5. Estructura del proyecto

Dentro de `src`, crea esta estructura:

```
src/
├── components/
│   ├── CourseCard.jsx
│   ├── CourseList.jsx
│   ├── SearchBar.jsx
│   └── Header.jsx
├── hooks/
│   └── useLocalStorage.js
├── services/
│   └── courseService.js
├── utils/
│   └── sanitize.js
├── App.jsx
└── main.jsx
```



└─ App.css

6. API que usaremos

Usaremos la API pública:

`https://jsonplaceholder.typicode.com/posts`

Como esta API devuelve “posts”, los simularemos como si fueran cursos.

Cada elemento tiene esta estructura JSON:

```
{
  "userId": 1,
  "id": 1,
  "title": "titulo del curso",
  "body": "descripcion del curso"
}
```

7. Crear servicio para consumir API

Crea el archivo:

`src/services/courseService.js`

Código:

```
import axios from "axios";

const API_URL = "https://jsonplaceholder.typicode.com/posts";

export const getCourses = async () => {
  try {
    const response = await axios.get(API_URL);

    return response.data.map((item) => ({
      id: item.id,
      title: item.title,
      description: item.body,
      teacherId: item.userId,
    }));
  } catch (error) {
    console.error("Error al obtener los cursos:", error);
    throw new Error("No fue posible cargar los cursos.");
  }
};
```

Explicación

Aquí separamos la lógica de conexión a la API del resto de la aplicación. Esto mejora el orden, la reutilización y la mantenibilidad.



Usamos:

```
await axios.get(API_URL)
```

para esperar la respuesta de la API antes de continuar.

8. Crear función básica de sanitización

Crea:

```
src/utils/sanitize.js
```

Código:

```
export const sanitizeText = (text) => {  
  if (typeof text !== "string") return "";  
  
  return text  
    .replace(/</g, "&lt;")  
    .replace(/>/g, "&gt;")  
    .trim();  
};
```

Explicación

Aunque React ya escapa texto renderizado por defecto, esta función ayuda a reforzar la validación básica y evita renderizar texto malicioso si viene desde una API externa.

Buena práctica: nunca usar `dangerouslySetInnerHTML` salvo que sea estrictamente necesario.

9. Crear Hook personalizado para LocalStorage

Crea:

```
src/hooks/useLocalStorage.js
```

Código:

```
import { useEffect, useState } from "react";  
  
export const useLocalStorage = (key, initialValue) => {  
  const [value, setValue] = useState(() => {  
    try {  
      const storedValue = localStorage.getItem(key);  
      return storedValue ? JSON.parse(storedValue) : initialValue;  
    } catch (error) {  

```



```
        console.error("Error leyendo localStorage:", error);
        return initialValue;
    }
});

useEffect(() => {
    try {
        localStorage.setItem(key, JSON.stringify(value));
    } catch (error) {
        console.error("Error guardando en localStorage:", error);
    }
}, [key, value]);

return [value, setValue];
};
```

Explicación

Este hook permite guardar información en el navegador.

Lo usaremos para mantener los favoritos aunque el usuario cierre o recargue la página.

Importante: no guardar contraseñas, tokens sensibles o datos personales críticos en localStorage.

10. Crear componente Header

Crea:

src/components/Header.jsx

Código:

```
const Header = () => {
    return (
        <header className="header">
            <h1>Gestor de Cursos React</h1>
            <p>SPA con API, LocalStorage, componentes y buenas prácticas de
seguridad.</p>
        </header>
    );
};

export default Header;
```

11. Crear componente SearchBar

Crea:

src/components/SearchBar.jsx

Código:

```
const SearchBar = ({ searchTerm, onSearchChange }) => {
  return (
    <div className="search-container">
      <label htmlFor="search">Buscar curso:</label>
      <input
        id="search"
        type="text"
        value={searchTerm}
        onChange={(event) => onSearchChange(event.target.value)}
        placeholder="Ejemplo: programación, seguridad, react..."
        maxLength={50}
      />
    </div>
  );
};

export default SearchBar;
```

Explicación

Este componente recibe datos mediante props:

```
searchTerm
onSearchChange
```

Así mantenemos el estado principal en App.jsx.

12. Crear componente CourseCard

Crea:

src/components/CourseCard.jsx

Código:

```
import { sanitizeText } from "../utils/sanitize";

const CourseCard = ({ course, isFavorite, onToggleFavorite }) => {
```



```
const safeTitle = sanitizeText(course.title);
const safeDescription = sanitizeText(course.description);

return (
  <article className="course-card">
    <h2>{safeTitle}</h2>

    <p>{safeDescription}</p>

    <small>Docente ID: {course.teacherId}</small>

    <button
      type="button"
      onClick={() => onToggleFavorite(course)}
      className={isFavorite ? "btn favorite" : "btn"}
    >
      {isFavorite ? "Quitar de favoritos" : "Agregar a favoritos"}
    </button>
  </article>
);
};

export default CourseCard;
```

13. Crear componente CourseList

Crea:

src/components/CourseList.jsx

Código:

```
import CourseCard from "../CourseCard";

const CourseList = ({ courses, favorites, onToggleFavorite }) => {
  if (courses.length === 0) {
    return <p className="message">No se encontraron cursos.</p>;
  }

  return (
    <section className="course-list">
      {courses.map((course) => {
        const isFavorite = favorites.some((fav) => fav.id === course.id);

        return (
          <CourseCard
            key={course.id}
            course={course}
            isFavorite={isFavorite}
            onToggleFavorite={onToggleFavorite}
          />
        );
      })}
    </section>
  );
};
```



```
};
```

```
export default CourseList;
```

14. Programar App.jsx

Reemplaza el contenido de:

src/App.jsx

por:

```
import { useEffect, useMemo, useState } from "react";
import "./App.css";

import Header from "../components/Header";
import SearchBar from "../components/SearchBar";
import CourseList from "../components/CourseList";

import { getCourses } from "../services/courseService";
import { useLocalStorage } from "../hooks/useLocalStorage";

function App() {
  const [courses, setCourses] = useState([]);
  const [searchTerm, setSearchTerm] = useState("");
  const [favorites, setFavorites] = useLocalStorage("favoriteCourses", []);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  const loadCourses = async () => {
    try {
      setLoading(true);
      setError("");

      const data = await getCourses();
      setCourses(data);
    } catch (error) {
      setError(error.message || "Ocurrió un error inesperado.");
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    loadCourses();
  }, []);

  const filteredCourses = useMemo(() => {
    const normalizedSearch = searchTerm.toLowerCase().trim();

    return courses.filter((course) =>
      course.title.toLowerCase().includes(normalizedSearch)
    );
  }, [courses, searchTerm]);
```




```
const handleToggleFavorite = (course) => {
  const exists = favorites.some((fav) => fav.id === course.id);

  if (exists) {
    const updatedFavorites = favorites.filter((fav) => fav.id !==
course.id);
    setFavorites(updatedFavorites);
    return;
  }

  setFavorites([...favorites, course]);
};

return (
  <main className="app">
    <Header />

    <section className="summary">
      <p>Total de cursos: {courses.length}</p>
      <p>Favoritos: {favorites.length}</p>
    </section>

    <SearchBar searchTerm={searchTerm} onSearchChange={setSearchTerm}
/>

    {loading && <p className="message">Cargando cursos...</p>}

    {error && (
      <div className="error">
        <p>{error}</p>
        <button type="button" onClick={loadCourses}>
          Reintentar
        </button>
      </div>
    )}

    {!loading && !error && (
      <CourseList
        courses={filteredCourses}
        favorites={favorites}
        onToggleFavorite={handleToggleFavorite}
      />
    )}
  </main>
);
}
```

```
export default App;
```

15. Programar estilos CSS

Reemplaza el contenido de:

src/App.css

por:

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background: #f4f6f8;
  color: #1f2937;
}

.app {
  width: min(1100px, 95%);
  margin: 0 auto;
  padding: 24px 0;
}

.header {
  background: #111827;
  color: white;
  padding: 24px;
  border-radius: 16px;
  margin-bottom: 20px;
}

.header h1 {
  margin: 0 0 8px;
}

.header p {
  margin: 0;
}

.summary {
  display: flex;
  gap: 16px;
  margin-bottom: 20px;
}

.summary p {
  background: white;
  padding: 12px 16px;
  border-radius: 12px;
```



```
    font-weight: bold;
}

.search-container {
  background: white;
  padding: 16px;
  border-radius: 12px;
  margin-bottom: 20px;
}

.search-container label {
  display: block;
  font-weight: bold;
  margin-bottom: 8px;
}

.search-container input {
  width: 100%;
  padding: 12px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  font-size: 16px;
}

.course-list {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 16px;
}

.course-card {
  background: white;
  padding: 18px;
  border-radius: 14px;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.06);
}

.course-card h2 {
  font-size: 18px;
  margin-top: 0;
  text-transform: capitalize;
}

.course-card p {
  line-height: 1.5;
}

.course-card small {
  display: block;
  margin-bottom: 12px;
  color: #475569;
}

.btn {
  width: 100%;
  border: none;
  padding: 10px;
```



```
border-radius: 8px;
background: #2563eb;
color: white;
cursor: pointer;
font-weight: bold;
}

.btn:hover {
  background: #1d4ed8;
}

.btn.favorite {
  background: #dc2626;
}

.btn.favorite:hover {
  background: #b91c1c;
}

.message {
  background: white;
  padding: 16px;
  border-radius: 12px;
}

.error {
  background: #fee2e2;
  color: #991b1b;
  padding: 16px;
  border-radius: 12px;
  margin-bottom: 20px;
}

.error button {
  padding: 8px 12px;
  border: none;
  background: #991b1b;
  color: white;
  border-radius: 8px;
  cursor: pointer;
}
```

16. Ejecutar aplicación

npm run dev

Prueba lo siguiente:

- Ver cursos cargados desde la API.
- Buscar cursos.
- Agregar cursos a favoritos.
- Recargar la página.
- Confirmar que los favoritos se mantienen.

17. Validar LocalStorage

Abre el navegador:

F12 → Application → Local Storage

Busca la clave:

`favoriteCourses`

Ahí deberías ver un arreglo JSON con los cursos favoritos.

18. Buenas prácticas de seguridad SPA aplicadas

En esta aplicación se aplican las siguientes medidas:

18.1 No guardar datos sensibles en LocalStorage

Correcto:

```
localStorage.setItem("favoriteCourses", JSON.stringify(favorites));
```

Incorrecto:

```
localStorage.setItem("token", jwtToken);  
localStorage.setItem("password", password);
```

18.2 Validar entradas del usuario

En el buscador usamos:

```
maxLength={50}
```

y normalizamos texto:

```
searchTerm.toLowerCase().trim();
```

18.3 Evitar HTML dinámico peligroso

No usar:

```
dangerouslySetInnerHTML
```

Ejemplo inseguro:

```
<div dangerouslySetInnerHTML={{ __html: course.description }} />
```

18.4 Manejo de errores

El servicio captura errores:

```
catch (error) {  
  console.error("Error al obtener los cursos:", error);  
  throw new Error("No fue posible cargar los cursos.");  
}
```

18.5 Separar responsabilidades

La aplicación está separada en:

```
components/  
services/  
hooks/  
utils/
```

Esto ayuda a mantener el código limpio y seguro.

OWASP es una referencia ampliamente usada para seguridad de aplicaciones web, y su Top 10 se utiliza como documento de concientización sobre riesgos críticos en aplicaciones.

19. Uso responsable de GitHub Copilot

Puedes usar Copilot para apoyar el desarrollo, pero no para copiar sin revisar.

Prompts recomendados

Ejemplo 1:



Genera un componente React reutilizable para mostrar una tarjeta de curso, usando props y evitando dangerouslySetInnerHTML.

Ejemplo 2:

Revisa este código React y sugiere mejoras de seguridad, legibilidad y manejo de errores.

Ejemplo 3:

Ayúdame a crear un hook personalizado para guardar datos no sensibles en localStorage.

Reglas importantes

- No aceptar código sin entenderlo.
- No subir claves, tokens ni credenciales.
- Revisar que no use funciones inseguras.
- Verificar licencias de librerías sugeridas.
- Validar que el código funcione realmente.

20. Análisis con SonarQube

SonarQube permite analizar calidad y seguridad del código. Su documentación indica que el scanner para NPM permite ejecutar análisis en proyectos JavaScript usando línea de comandos o npx. (docs.sonarsource.com)

20.1 Crear archivo sonar-project.properties

En la raíz del proyecto crea:

```
sonar-project.properties
```

Contenido:

```
sonar.projectKey=gestor-cursos-react
sonar.projectName=Gestor Cursos React
sonar.projectVersion=1.0

sonar.sources=src
sonar.exclusions=node_modules/**,dist/**

sonar.sourceEncoding=UTF-8
```

20.2 Ejecutar SonarQube con Docker

Crea un archivo:

```
docker-compose.yml
```

Contenido:

```
services:
  sonarqube:
    image: sonarqube:community
    container_name: sonarqube-react
    ports:
      - "9000:9000"
    environment:
      - SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true
```

Ejecuta:

```
docker compose up -d
```

Abre:

```
http://localhost:9000
```

Credenciales iniciales comunes:

```
Usuario: admin
Clave: admin
```

Luego SonarQube puede pedir cambiar la contraseña.

20.3 Ejecutar análisis

Desde la raíz del proyecto:

```
npx @sonar/scan
```

Si necesitas indicar token:

```
npx @sonar/scan \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=TU_TOKEN
```




En Windows PowerShell:

```
npx @sonar/scan `
  -Dsonar.host.url=http://localhost:9000 `
  -Dsonar.token=TU_TOKEN
```

21. Prueba final esperada

El estudiante debe demostrar que:

- La app carga datos desde API.
- Los datos se manejan como JSON.
- Se usa `async/await`.
- La interfaz está dividida en componentes.
- Los favoritos se guardan en `localStorage`.
- Existe manejo de errores.
- No se usa `dangerouslySetInnerHTML`.
- Se aplican validaciones básicas.
- Se utilizó IA como apoyo, no como reemplazo del análisis.
- Se ejecutó o documentó revisión con SonarQube.

22. Una vez terminados los pasos anteriores y con la ayuda de la IA

Para subir el nivel, agrega:

Desafío 1: filtro por docente

Crear un select para filtrar por `teacherId`.

Desafío 2: contador por favoritos

Mostrar cuántos cursos favoritos tiene cada docente.

Desafío 3: modo oscuro

Guardar preferencia en `localStorage`.

Desafío 4: cambio de fetch a axios

Crear una versión alternativa de `courseService.js` usando `fetch`.

Ejemplo:

```
const API_URL = "https://jsonplaceholder.typicode.com/posts";
```

```
export const getCoursesWithFetch = async () => {
  const response = await fetch(API_URL);

  if (!response.ok) {
    throw new Error("Error al consumir la API.");
  }

  const data = await response.json();

  return data.map((item) => ({
    id: item.id,
    title: item.title,
    description: item.body,
    teacherId: item.userId,
  }));
};
```

23. Entregables

El estudiante debe entregar:

1. Código fuente del proyecto.
2. Capturas de la aplicación funcionando.
3. Captura de localStorage con favoritos guardados.
4. Breve explicación de componentes creados.
5. Evidencia de uso de API.
6. Evidencia o reporte de SonarQube.
7. Reflexión breve sobre uso responsable de IA.

24. Criterios de evaluación

Criterio	Puntaje
Creación correcta del proyecto React	10
Uso de componentes reutilizables	15
Consumo de API con async/await	15
Manejo correcto de JSON	10
Uso funcional de LocalStorage	15
Buenas prácticas de seguridad SPA	15
Uso responsable de IA	10
Revisión con SonarQube	10
Total	100